

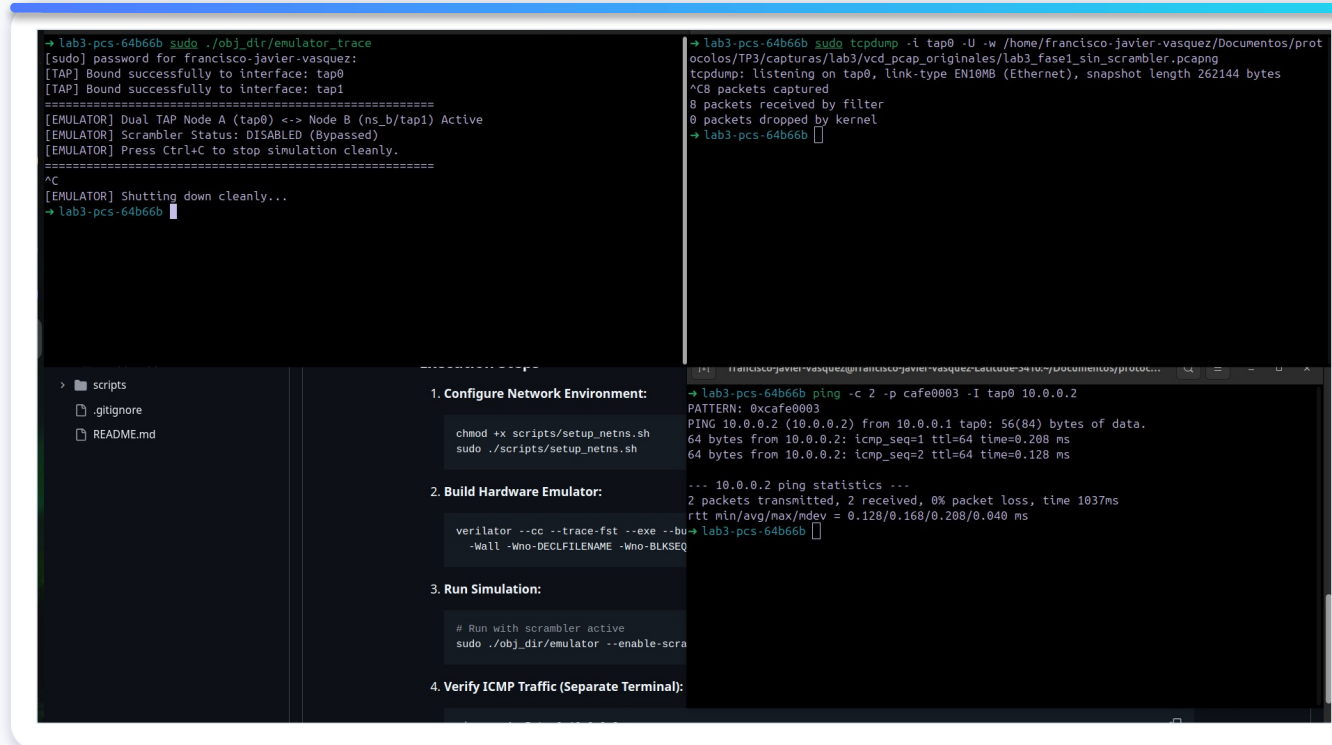
TP3 · Communication Protocols

Lab 3 — PCS 64b/66b Encoding & Scrambling

Vasquez Curetti, Francisco Javier

Testbench: emulator, capture & traffic generation

TP3 · Lab 3



```
→ lab3-pcs-64b66b sudo ./obj_dir/emulator_trace
[sudo] password for francisco-javier-vasquez:
[TAP] Bound successfully to interface: tap0
[TAP] Bound successfully to interface: tap1
=====
[EMULATOR] Dual TAP Node A (tap0) <-> Node B (ns_b/tap1) Active
[EMULATOR] Scrambler Status: DISABLED (Bypassed)
[EMULATOR] Press Ctrl+C to stop simulation cleanly.
=====
^C
[EMULATOR] Shutting down cleanly...
→ lab3-pcs-64b66b

→ lab3-pcs-64b66b sudo tcpdump -i tap0 -U -w /home/francisco-javier-vasquez/Documentos/prot
ocolos/TP3/capturas/lab3/vcd_pcap_originales/lab3_fase1_sin_scrambler.pcapng
tcpdump: listening on tap0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C0 packets captured
0 packets received by filter
0 packets dropped by kernel
→ lab3-pcs-64b66b

→ lab3-pcs-64b66b ping -c 2 -p cafe0003 -I tap0 10.0.0.2
PATTERN: 0xcafe0003
PING 10.0.0.2 (10.0.0.2) from 10.0.0.1 tap0: 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.208 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.128 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1037ms
rtt min/avg/max/mdev = 0.128/0.168/0.208/0.040 ms
→ lab3-pcs-64b66b
```

Phase 1 (scrambler off): emulator_trace (default, no --enable-scrambler) + tcpdump -i tap0 + ping -c 2 -p cafe0003 -I tap0 10.0.0.2

Wireshark — 8 packets: ARP, two Echo pairs

The image shows a Wireshark packet capture with 8 packets. The packet list pane shows the following details:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	02:ca:ee:44:4a:fa	Broadcast	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
2	0.000090	b6:cb:dc:76:cc:85	02:ca:ee:44:4a:fa	ARP	60	10.0.0.2 is at b6:cb:dc:76:cc:85
3	0.000094	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x8e25, seq=1/256, ttl=64 (reply in 4)
4	0.000196	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply id=0x8e25, seq=1/256, ttl=64 (request in 3)
5	1.037388	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x8e25, seq=2/512, ttl=64 (reply in 6)
6	1.037502	10.0.0.2	10.0.0.1	ICMP	98	Echo (ping) reply id=0x8e25, seq=2/512, ttl=64 (request in 5)
7	5.133371	b6:cb:dc:76:cc:85	02:ca:ee:44:4a:fa	ARP	60	Who has 10.0.0.1? Tell 10.0.0.2
8	5.133381	02:ca:ee:44:4a:fa	b6:cb:dc:76:cc:85	ARP	42	10.0.0.1 is at 02:ca:ee:44:4a:fa

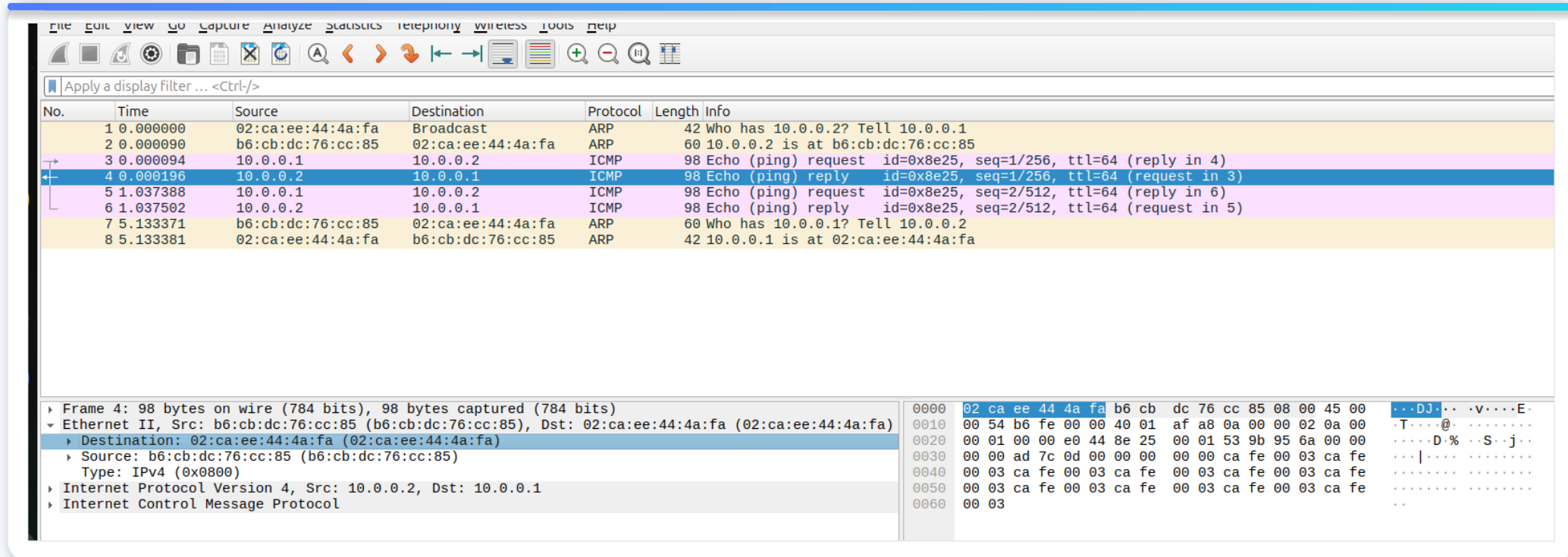
The packet details pane for packet 3 (Frame 3) shows the following structure:

- Frame 3: 98 bytes on wire (784 bits), 98 bytes captured (784 bits)
- Ethernet II, Src: 02:ca:ee:44:4a:fa (02:ca:ee:44:4a:fa), Dst: b6:cb:dc:76:cc:85 (b6:cb:dc:76:cc:85)
 - Destination: b6:cb:dc:76:cc:85 (b6:cb:dc:76:cc:85)
 - Source: 02:ca:ee:44:4a:fa (02:ca:ee:44:4a:fa)
 - Type: IPv4 (0x0800)
- Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
- Internet Control Message Protocol

The packet bytes pane shows the raw data in hexadecimal and ASCII:

```
0000 b6 cb dc 76 cc 85 02 ca ee 44 4a fa 08 00 45 00 ...v...DJ...E
0010 00 54 e3 5a 40 00 40 01 43 4c 0a 00 00 01 0a 00 .T.Z@. CL.....
0020 00 02 08 00 d8 44 8e 25 00 01 53 9b 95 6a 00 00 ....D.%..S..j..
0030 00 00 ad 7c 0d 00 00 00 00 00 ca fe 00 03 ca fe ...|.....
0040 00 03 ca fe 00 03 ca fe 00 03 ca fe 00 03 ca fe .....
0050 00 03 ca fe 00 03 ca fe 00 03 ca fe 00 03 ca fe .....
0060 00 03 ..
```

Wireshark — Echo Reply: same structure, reverse direction



Src/Dst swapped · ICMP type=0 (Echo Reply) · same cafe0003 pattern in the payload.

64b/66b encoding: the Sync Header decides everything first

Sync[1:0]	Classification	Description
2'b01	Data Block	Pure 64-bit payload (8 octets /D0.../D7/), no Block Type Field
2'b10	Control + Data	Combination of Data and Control bytes. Block Type field indicates the boundary.

hdr = 2'b01 when `cgmii_txc == 0x00` (all 8 lanes are data); **2'b10** in any other case (at least one control lane).

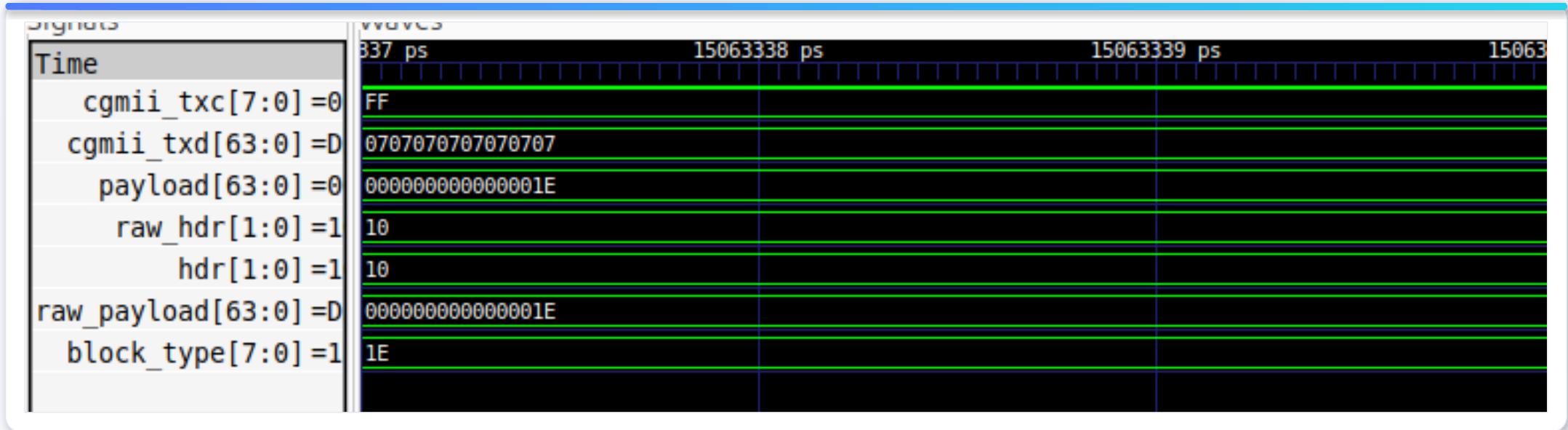
Block Type Field — IEEE 802.3 Clause 49

Input Data	S y n c	Block Payload									
Bit Position:	0 1 2	6									
Data Block Format:											
D ₀ D ₁ D ₂ D ₃ /D ₄ D ₅ D ₆ D ₇	01	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇		
Control Block Formats:		Block Type Field									
C ₀ C ₁ C ₂ C ₃ /C ₄ C ₅ C ₆ C ₇	10	0x1E	C ₀	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	
C ₀ C ₁ C ₂ C ₃ /O ₄ D ₅ D ₆ D ₇	10	0x2D	C ₀	C ₁	C ₂	C ₃	O ₄	D ₅	D ₆	D ₇	
C ₀ C ₁ C ₂ C ₃ /S ₄ D ₅ D ₆ D ₇	10	0x33	C ₀	C ₁	C ₂	C ₃		D ₅	D ₆	D ₇	
O ₀ D ₁ D ₂ D ₃ /S ₄ D ₅ D ₆ D ₇	10	0x66	D ₁	D ₂	D ₃	O ₀		D ₅	D ₆	D ₇	
O ₀ D ₁ D ₂ D ₃ /O ₄ D ₅ D ₆ D ₇	10	0x55	D ₁	D ₂	D ₃	O ₀	O ₄	D ₅	D ₆	D ₇	
S ₀ D ₁ D ₂ D ₃ /D ₄ D ₅ D ₆ D ₇	10	0x78	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇		
O ₀ D ₁ D ₂ D ₃ /C ₄ C ₅ C ₆ C ₇	10	0x4B	D ₁	D ₂	D ₃	O ₀	C ₄	C ₅	C ₆	C ₇	
T ₀ C ₁ C ₂ C ₃ /C ₄ C ₅ C ₆ C ₇	10	0x87		C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	
D ₀ T ₁ C ₂ C ₃ /C ₄ C ₅ C ₆ C ₇	10	0x99	D ₀		C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	
D ₀ D ₁ T ₂ C ₃ /C ₄ C ₅ C ₆ C ₇	10	0xAA	D ₀	D ₁		C ₃	C ₄	C ₅	C ₆	C ₇	
D ₀ D ₁ D ₂ T ₃ /C ₄ C ₅ C ₆ C ₇	10	0xB4	D ₀	D ₁	D ₂		C ₄	C ₅	C ₆	C ₇	
D ₀ D ₁ D ₂ D ₃ /T ₄ C ₅ C ₆ C ₇	10	0xCC	D ₀	D ₁	D ₂	D ₃		C ₅	C ₆	C ₇	
D ₀ D ₁ D ₂ D ₃ /D ₄ T ₅ C ₆ C ₇	10	0xD2	D ₀	D ₁	D ₂	D ₃	D ₄		C ₆	C ₇	
D ₀ D ₁ D ₂ D ₃ /D ₄ D ₅ T ₆ C ₇	10	0xE1	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅		C ₇	
D ₀ D ₁ D ₂ D ₃ /D ₄ D ₅ D ₆ T ₇	10	0xFF	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆		

Table 49-1 — Encoding control characters within the PCS

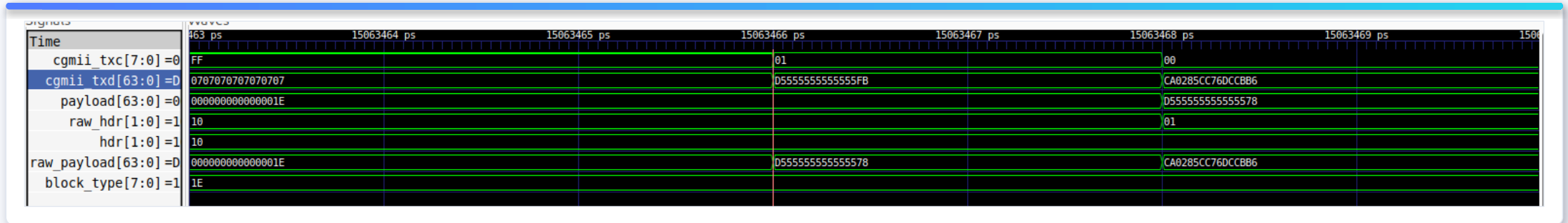
Table 49-1—Control codes

Control character	Notation	XGMII Control Code	10GBASE-R Control Code	10GBASE-R O Code	8B/10B Code ^a
idle	/I/	0x07	0x00		K28.0 or K28.3 or K28.5
LPI	/LI/	0x06	0x06		K28.0 with D20.5 in one row or K28.3 or K28.5 with D20.5 in one row ^b
start	/S/	0xFB	Encoded by block type field		K27.7
terminate	/T/	0xFD	Encoded by block type field		K29.7
error	/E/	0xFE	0x1E		K30.7
Sequence ordered set	/Q/	0x9C	Encoded by block type field plus O code	0x0	K28.4
reserved0	/R ^c /	0x1C	0x2D		K28.0
reserved1		0x3C	0x33		K28.1
reserved2	/A/	0x7C	0x4B		K28.3
reserved3	/K/	0xBC	0x55		K28.5



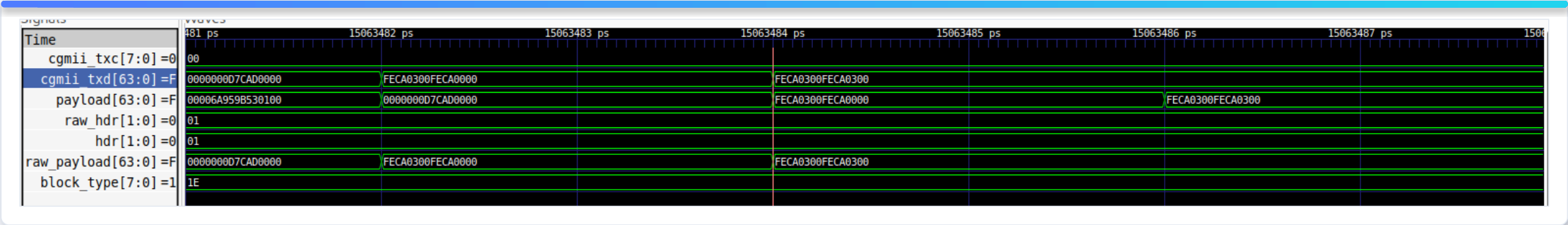
Compression: /I/ (0x07) → 0x00

Framing: sync = 10 ; BTF = 0x1E → All control characters



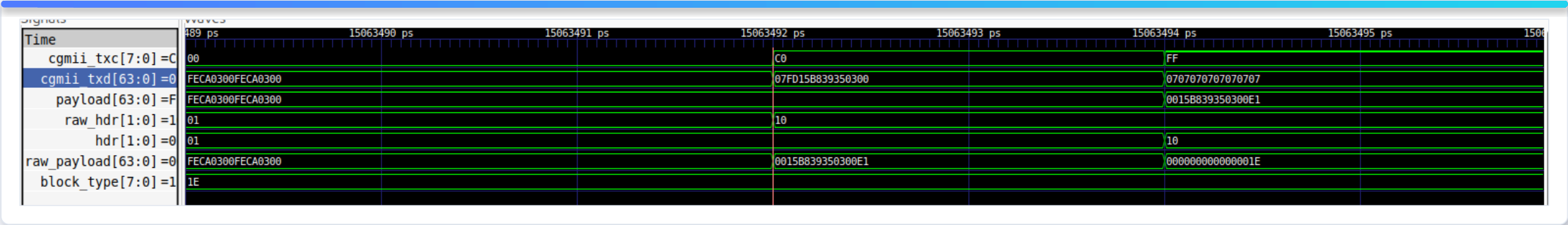
Start (/S/): (0xFB) is dropped entirely – its presence is now implicit in sync=10 and BTF=0x78
Preamble + SFD: marked as data, so they pass through into raw_payload unchanged

Next block: txc=0x00 (no control lanes) → hdr flips to 01, no BTF at all



txc=00 → **hdr=2'b01** (pure Data, no BTF). raw_payload=FECA0300FECA0000 / FECA0300FECA0300 — the same cafe0003 seen in the Wireshark hex dump.

GTKWave — Terminate /T/ & BTF selection



/T/=0xFD landed on lane 6 → txc=0xC0, sync= 10, BTF=0xE1. See Table in Slide 6

Self-synchronizing scrambler: $G(x) = x^{58} + x^{39} + 1$

1

Why to scramble? High-speed serial links require sufficient bit-transition density for clock recovery. Idle, Start, and Terminate blocks sometimes are the worst case: short, repetitive, and filled with zeros patterns.

2

How to encode? A LFSR — $G(x) = x^{58} + x^{39} + 1$ — XORs the 64-bit payload against a pseudo-random sequence derived from a fixed seed shared by both ends.

3

Sync header is not scrambled — it must stay as it is so the receiver can manage to achieve block alignment before descrambling

GTKWave — Scrambler active: payload randomization

Time	163 ps	10294164 ps	10294165 ps	10294166 ps	10294167 ps	10294168 ps	10294169 ps
cgmii_txc[7:0]=0	00						
cgmii_txd[63:0]=F	0000000B82B30000	FECA0300FECA0000		FECA0300FECA0300			
hdr[1:0]=0	01						
payload[63:0]=9	CAA83350E5586F99	ACCE46A3953FC9CC		96AE4CCC17B0AEED		E5D9627899B293B0	
raw_hdr[1:0]=0	01						
raw_payload[63:0]=F	0000000B82B30000	FECA0300FECA0000		FECA0300FECA0300			
enable_scrambler=1							
descram_payload[63:0]=F	00006A95BBB90100	0000000B82B30000		FECA0300FECA0000		FECA0300FECA0300	

Same instant: raw_payload=FECA0300FECA0000 vs payload=ACCE46A3953FC9CC. The first is recognizable (from cafe0003), the second shows no visible pattern. Sync header is not scrambled.

descram_payload matches **raw_payload**. Node B's LFSR correctly reconstructs the payload before scrambling.